

MINILAB_CLI_GLOBAL_PLAN.md — v0.3.1

Split Labour by Talent / Maximum Automation UX

Status: Plano global refinado da CLI

Escopo: orientar o crescimento do `minilab-cli` sem aumentar superfície antes da hora

Tese: automação máxima é dar a cada ente o menor mundo suficiente para agir bem

Próximo gate: Gate 4.2 — Receipt Machinery Compatibility Probe

1. Tese central

A CLI do Minilab é o **primeiro e único corpo institucional executável**.

A gramática é LogLine.

A implementação institucional é Rust.

As crates são órgãos.

SDKs e clouds são maquinaria alugada.

Receipts são memória.

Dan assina consequência.

One grammar.

One executable institutional body.

Many machines.

Receipts everywhere.

A CLI não é “onde tudo acontece”.

A CLI é onde tudo entra, ganha forma, é testado, recebe limite, chama a maquinaria certa e deixa recibo.

The CLI coordinates machinery.

It does not become the machinery.

2. Automação máxima

Automação máxima não é remover humanos nem deixar agentes fazerem tudo.

Automação máxima é **ergonomia máxima para todos os entes da instituição**.

Cada ente deve receber a melhor UX possível para o seu talento.

Humans decide.

LLMs translate.

Mini-LLMs classify.

CLI materializes.

Scripts repeat.

LogLine judges.

Crates perform specialized machinery.

Tower authorizes power.

LABs execute.

Receipts remember.

Ou, em forma curta:

Split labour by talent.

Adopt organs by proof.

Keep the CLI as mouth, not stomach.

Receipts remember what actually happened.

3. Divisão de trabalho por talento

Humano

Talento:

direção

gosto

risco

autoridade

consequência

Melhor UX:

decisão clara

risco explícito
opções pequenas
evidência resumida
nenhum grep
nenhuma caça a arquivo

Humano decide consequência.
Humano não revisa lixo.

LLM grande

Talento:

ambiguidade
síntese
nomeação
planejamento
comparação
explicação

Melhor UX:

context-pack curto
workorder explícito
ghosts declarados
arquivos relevantes
probes obrigatórios
critério de aceite
limite de escopo

LLM caro não descobre o workspace.
LLM caro recebe um caso.

Mini-LLM

Talento:

classificar
preencher slots
resumir receipts

baixar linguagem natural para ato pequeno
escolher entre estados fechados

Melhor UX:

gramática curta
9 slots
pouco contexto
uma tarefa por vez
nenhuma obrigação de fingir verdade
nenhuma execução

Mini-LLM não precisa de liberdade infinita.
Precisa de superfície calma.

Operador

Talento:

orquestrar
inspecionar
rodar probes
ler diffs
decidir próximo patch

Melhor UX:

um comando
um status
um diff
um receipt
um next
erro classificável

Operador não é babá de agente.

Script determinístico

Talento:

repetição barata

varredura
contagem
cargo
grep
diff
hash
scan

Melhor UX:

input
output
exit 0/1
log
reprodutibilidade

Script não precisa gostar.
Script precisa rodar.

CLI

Talento:

materializar
validar
chamar crates
rodar probes
emitir receipts
detectar drift
estruturar workorders

Melhor UX:

args claros
exit code honesto
stdout JSON quando necessário
stderr humano quando útil
receipt path
nenhum estado oculto

A CLI fala.
A CLI não deve virar todos os órgãos.

Crates

Talento:

semântica especializada
ledger
hash
gate
runtime
gateway
proofpack
runner
policy

Melhor UX:

API pequena
cargo check
doctor
smoke
adoption profile
activation receipt
ownership claro

Crate não entra por entusiasmo.
Crate entra por rito.

MCP LLM / conectores

Talento:

chamar tools
integrar sistemas externos
expor superfícies

Melhor UX:

schemas pequenos
tools idempotentes
erros estruturados

side effects explícitos
receipts linkáveis

Conector não define lei.
Conector transporta ato.

LAB

Talento:

trabalho pesado
execução local
diagnóstico
inference
jobs demorados

Melhor UX:

job claro
capability declarada
runner identity
heartbeat
receipt de execução

LAB executa.
LAB não governa.

4. Invariants

I1 — Natural language never executes

Linguagem natural pode virar intenção, pergunta, candidato, rejeição ou ghost.

Não pode virar execução.

I2 — Todo comando relevante projeta ato LogLine-shaped

Espinha mínima:

who
did
this
when
confirmed_by
if_ok
if_doubt
if_not
status

I3 — Nenhum ato relevante sem probe ou receipt path

intent
→ act
→ probe
→ evidence
→ receipt

Comando sem caminho de prova é gesto.

I4 — No closure without evidence

Provider success não é closure.
Cargo green não é produto.
Output bonito não é verdade.

I5 — Estados não colapsam

generated ≠ run
run ≠ passed
passed ≠ reviewed
reviewed ≠ production-observed

I6 — if_doubt never becomes hidden ok

Dúvida é estado de primeira classe.

Se falta evidência, o caminho é:

GHOST
ASK
Challenge
simulate
suspend
manual review

Nunca ok silencioso.

I7 — Reuse-first, custom-last

Antes de escrever motor próprio:

Existe crate?
Existe SDK?
Existe protocolo?
Existe implementação anterior nossa?
Existe API pública usável?
Existe cargo check?
Existe smoke?

Custom code só entra quando uma invariante LogLine/Minilab exige.

I8 — CLI coordinates; crates own machinery

A CLI pode orquestrar, chamar, adaptar e registrar.

A CLI não deve virar:

runtime paralelo
ledger paralelo
gate paralelo
gateway paralelo
proofpack paralelo

I9 — Repo, config, secrets e data não se misturam

repo descreve
config instancia
secrets autorizam
data registra
receipts provam

I10 — Receipts beat stories

Todo avanço importante precisa dizer:

o que provou
o que não provou
qual escopo
qual hash
qual evidência
qual ghost
qual próximo gate

5. Boundaries

CLI vs LogLine

A CLI fala LogLine.
A CLI não inventa LogLine.

LogLine = grammar / law
CLI = executable institutional body

CLI vs Runtime

A CLI chama runtime/canon.

Runtime julga admissibilidade, walk, digest, lifecycle e consequência.

CLI vs Tower

A CLI prepara.
Tower autoriza poder operacional protegido.

CLI may prepare.
Tower authorizes protected power.

CLI vs Cloudflare / Gateway

Cloudflare é porta.
Gateway é projeção.
Nenhum dos dois é trono.

Gateway projects CLI grammar.
Gateway does not define law.

CLI vs SDKs / conectores

SDK entra por Adoption Lane:

idea
→ manifest
→ inspect
→ doctor
→ smoke
→ adopt
→ activate
→ receipt

Nada de **cargo add** com fé.

CLI vs LLM

LLM pode propor, explicar, estruturar, revisar e criticar.

LLM não executa, não despacha, não muta DB, não declara verified e não assina consequência.

Receipt semântico vs envelope de armazenamento

Não trocar `receipt_hash` por entusiasmo.

LIP-0003 `receipt_hash`
= identidade semântica do receipt

`json_atomic` / BLAKE3
= candidato a storage CID / envelope hash

Ed25519
= candidato a assinatura de envelope / ledger entry

ubl-ledger
= candidato a transcript / append / storage

O hash semântico atual permanece dono da memória já emitida.

6. Ofícios da CLI

A CLI tem cinco ofícios.

1. Grammar Surface

Transforma intenção em ato estruturado.

Comandos atuais/futuros:

`minilab doctrine`
`minilab operations`
`minilab workorder new`
`minilab workorder inspect`
`minilab workorder to-logline`
`minilab logline canon status`
`minilab logline walk`

2. Evidence Surface

Verifica, lista, compara, registra e prova.

Comandos atuais/futuros:

minilab receipts status
minilab receipts list
minilab receipts verify
minilab receipts doctor-machinery
minilab receipts trace

3. Adoption Surface

Faz órgãos entrarem com rito.

Comandos futuros:

minilab integration inspect
minilab integration doctor
minilab integration smoke
minilab integration adopt
minilab integration activate

4. Projection Surface

Projeta a gramática para outros entes.

Comandos futuros:

minilab workorder context-pack
minilab workorder render-agent-prompt
minilab gateway status
minilab gateway mcp status

5. Operations Surface

Só cresce depois de gate, ledger e adoption.

Comandos futuros:

minilab jobs ...
minilab lab ...
minilab expedition ...
minilab incident ...
minilab release ...
minilab snapshot ...

Estes não são prioridade agora.

7. Comando mínimo por operação

O padrão desejado é:

status
inspect
doctor
probe
receipt

Forma:

minilab <operation> status
minilab <operation> inspect <subject>
minilab <operation> doctor <subject>
minilab <operation> probe <subject>
minilab <operation> receipt <subject>

Refinamento importante:

Nem toda operação precisa expor os cinco verbos agora.
Toda operação nova deve declarar quais verbos existem
e quais ficam ghosted.

Skeleton honesto é permitido.
Fake implementation não.

8. Estado atual provado

A cadeia mínima provada é:

workorder.created
→ workorder.draft_projected
→ logline.walk_probed

Gates fechados:

Gate 0:

Rust-native CLI baseline

Gate 3:

local receipt discovery + scoped verification

Gate 3.1:

workorder.created LIP-0003 receipt

Gate 3.2:

workorder.draft_projected LIP-0003 receipt

Gate 4:

local runtime walk probe

Isso prova:

a boca existe

o cartório local lê

a boca emite receipts

tamper falha

o juiz caminhou um candidato

Isso não prova:

authoritative ledger sink

gate/admission

Tower

Gateway

Cloudflare

Supabase

LAB dispatch

production observation

9. Ownership map

Canon / grammar

Dono natural:

logline canon

logline-status

logline-who

Usar para:

9-slot tuple

LogLine shape

canonical digest

result_hash

receipt_hash

run_with_context

lifecycle

Regra:

minilab-cli não reimplementa canon.

Receipts

Dono semântico atual:

logline-status

Candidatos a maquinaria de armazenamento/verificação:

ubl-ledger

json_atomic

llv-core

llv-index

Decisão:

LIP-0003 receipt_hash não muda.

Storage/envelope pode evoluir ao redor dele.

Gate / admission

Candidatos:

tdln-gate

constitution-gate

constitutional-runtime

Direção provável:

constitution-gate = lei / taxonomia constitucional
tdln-gate = motor operacional de preflight/decide

Não implementar Gate 5 antes do doctor de receipts.

Gateway

Candidatos:

ubl-mcp
LogLineOS-v8 gateway/router/runtime/validators
constitutional-runtime minilab-api

Direção:

Gate 6+.
Gateway é projeção da CLI, não instituição paralela.

LAB / jobs

Candidatos:

ubl-office
ecosystem-jobs
ecosystem-executors
minilab-runner

Direção:

Gate 7+.
LAB executa depois de gate/ledger.

ProofPack / portable proof

Candidatos:

tdln-core
tdln-bundle
tdln-wasm
tdln-verify

proof-runtime
epistemic-storage
proof-federation
worker-abi

Direção:

Gate 8+.

Não misturar ProofPack/DiamondCard com LIP-0003 sem adoption profile.

Manager plane

Candidato:

manager-plane

Status:

quarantine

Motivo:

shell_exec / default_ACK exigem sandbox e worker boundary antes de adoção.

10. Roadmap refinado

Done

Gate 0 Rust-native CLI baseline
Gate 3 Local receipt discovery + verify
Gate 3.1 workorder.created receipt
Gate 3.2 workorder.draft_projected receipt
Gate 4 Local runtime walk probe

Now

Gate 4.2 — Receipt Machinery Compatibility Probe

Objetivo:

Testar se crates de ledger/hash/verificação aceitam os receipts atuais

sem trocar sua semântica.

Comando:

minilab receipts doctor-machinery <receipt-path>

Rodar em:

receipts/workorders/round-0004/created.receipt.json

receipts/logline/round-0005/walk_probed.receipt.json

O doctor deve responder:

```
{
  "input_profile": "LIP-0003",
  "semantic_hash_owner": "logline-status",
  "receipt_hash_changed": false,
  "checks": {
    "current_verify": "pass",
    "json_atomic_envelope": "pass|fail|not_applicable",
    "ubl_ledger_temp_append": "pass|fail|not_applicable",
    "lllv_core_probe": "pass|fail|not_applicable"
  },
  "adoption_decision": "safe_to_adopt|adapter_required|do_not_adopt_yet",
  "ghosts": []
}
```

Hard rules:

- não mudar receipt_hash
- não reescrever receipts existentes
- não persistir em ledger real
- usar tempdir
- não iniciar Gate 5
- não iniciar Gateway
- não iniciar Supabase
- não iniciar LAB
- não criar novo receipt canon

Later

Gate 5:

local gate/admission using tdlm-gate / constitution-gate

Gate 6:

gateway projection using ubl-mcp / LogLineOS-v8

Gate 7:

LAB runner observation using ubl-office / ecosystem

Gate 8:

ProofPack / snapshot / release verification

Gate 9:

LogLine-governed build

11. Token austerity

Tokens ficaram caros. Isso é bom para a arquitetura.

Regra:

No expensive agent without a workorder.

No repo-wide context without a context-pack.

No implementation request without probes.

No repeated explanation without receipts.

O agente caro não pensa o projeto.

O agente caro recebe um ato.

Futuro comando:

minilab workorder context-pack <slug>

Context pack contém:

goal

relevant files

boundaries

ghosts

commands

acceptance probes

expected diff

receipt format

budget

12. Anti-patterns

Parar imediatamente se aparecer:

CLI grows into runtime
CLI grows into ledger
CLI grows into gate
CLI grows into gateway
LLM behind dispatcher
Cloudflare defines semantics
Supabase becomes truth
receipt_hash changes silently
default_ACK
shell_exec without worker boundary
dashboard as center
natural language as action
provider success as closure
scaffold presented as implementation

13. Acceptance format

Todo avanço deve responder:

Intent:

Qual é o objetivo sem encolher?

World:

Quais arquivos/crates/sistemas toca?

Ghosts:

O que ainda é inferred, unverified, duplicado ou arriscado?

Action:

Qual foi a menor mudança real?

Receipt:

Qual comando/probe prova?

State:

draft | proposed | implemented-unverified | evidence-captured | scoped-commit | ghosted | rejected

Desk Clean:

O que o próximo humano/modelo/runtime precisa saber?

14. Definition of done

Nenhum scoped-commit sem:

cargo check/build se Rust
comportamento exercitado
receipt gerado ou verificado
tamper/failure path quando relevante
escopo honesto
ghosts declarados
nenhum fake implementation
nenhuma troca silenciosa de semântica

15. Próximo commit recomendado

Título:

Probe receipt machinery compatibility without changing receipt canon

Escopo:

Add non-destructive receipt machinery doctor.

Arquivos prováveis:

crates/minilab-cli/src/commands/receipts.rs
docs/GATE4_2_RECEIPT_MACHINERY_DOCTOR.md
run-g42-receipt-machinery-doctor.sh

Probes:

cargo check --workspace
cargo build -p minilab-cli

minilab receipts doctor-machinery \
receipts/workorders/round-0004/created.receipt.json

```
minilab receipts doctor-machinery \  
  receipts/logline/round-0005/walk_probed.receipt.json
```

```
minilab receipts verify \  
  receipts/workorders/round-0004/created.receipt.json
```

```
minilab receipts verify \  
  receipts/logline/round-0005/walk_probed.receipt.json
```

E confirmar:

```
tampered fixtures still fail  
existing receipts unchanged  
no real ledger persistence  
no receipt_hash change
```

16. Final sentence

Minilab CLI is the Rust-native executable institution
that gives each actor the smallest world in which it can act well.

Split labour by talent.
Adopt organs by proof.
Keep the CLI as mouth, not stomach.
Let receipts remember what actually happened.

Ou em português:

A CLI é a boca institucional do Minilab.
Ela fala LogLine, chama os órgãos certos, roda probes e deixa recibos.

Ela não vira todos os órgãos.
Ela não troca a lei do carimbo.
Ela não transforma dúvida em ok.

A automação máxima é cada ente fazendo só aquilo que faz bem,
sob uma gramática comum,
com receipts fechando o que aconteceu.